

Submission Pack

Project Master, Agent Spec, Build Journal, and Playbook

Prepared as an office-ready planning package for the AI Engineer assessment. No implementation code has been started.

Document 1: Project Master File

See separate Project_Master_File.docx for the formatted master file. Key decision: local-first extraction and drafting with Gemini/Claude BYOK paid fallback when quality gates fail.

Document 2: Agent-Ready Spec File

Implementation instructions for coding agents

Agent-Ready Spec File

Project: Local-First Document Understanding, Grounded Drafting, and Edit Improvement

Version: 1.0

Status: Ready for implementation planning; no coding has started.

0. Prime Directive for Agents

Build the system task-by-task without asking the owner for routine decisions. Use the defaults in this spec. Ask the owner only when a decision affects legal risk, data retention, paid API usage behavior, or the demo scope.

The system must run locally first. It must call paid vision models only when the local path fails quality gates or when the operator explicitly requests paid fallback. Paid models must use BYOK keys supplied by the user.

1. Problem Statement

The office receives messy legal-style documents: scanned PDFs, handwritten notes, screenshots, photos, partially legible records, and inconsistent forms. The system must ingest them, extract usable content, ground later answers in source evidence, generate first-pass drafts, and improve from operator edits.

2. Target Users

- Intake operator: uploads documents and checks quality.
- Reviewer/analyst: reviews source evidence and edits output.
- Admin: configures local and BYOK model providers.

3. In-Scope MVP

- Upload PDF, PNG, JPG, JPEG, TIFF, DOCX, TXT.
- Render PDFs to images page-by-page.
- Preprocess images: deskew, denoise, rotate, crop margins, contrast normalization.
- Local OCR with confidence scores.
- Structured extraction: document title, dates, parties, addresses, amounts, deadlines, named entities, page references.
- Local embeddings and vector retrieval.
- Grounded draft generation with citations to source document, page, and text span.
- Quality dashboard visible to user.
- BYOK Gemini and Claude vision fallback configuration.
- Edit capture and prompt/template improvement loop.
- Test fixtures, unit tests, integration tests, and end-to-end demo flow.

4. Out of Scope for MVP

- Final legal advice.
- E-signatures or court filing.
- Native mobile app.
- Multi-tenant billing.
- Training a custom OCR model.
- Long-term storage of raw sensitive documents unless explicitly enabled.

5. Architecture

Upload UI

- > File Intake Service
- > PDF/Image Normalizer
- > Local OCR + Local Vision/LLM Adapter
- > Quality Gate
 - > if pass: proceed
 - > if fail and BYOK enabled: paid vision fallback per failed page/field
 - > if fail and no BYOK: warn user and continue only with degraded flag
- > Document Chunker
- > Local Embedding Store
- > Retrieval Engine
- > Grounded Draft Generator
- > Citation Verifier
- > Review/Edit UI
- > Edit Learning Store
- > Metrics Dashboard

6. Provider Strategy

6.1 Provider Interface

Create a common provider interface:

```
interface VisionTextProvider {  
    name: string;  
    isAvailable(): Promise<boolean>;  
    extract(input: ProviderInput): Promise<ProviderExtractionResult>;  
}  
  
interface DraftProvider {  
    name: string;  
    isAvailable(): Promise<boolean>;  
    generate(input: DraftInput): Promise<DraftResult>;  
}
```

6.2 Default Provider Order

1. local_ocr
2. local_vision_llm
3. gemini_byok
4. claude_byok

6.3 BYOK Environment Variables

GEMINI_API_KEY=

GEMINI_VISION_MODEL=gemini-3-flash

ANTHROPIC_API_KEY=

CLAUDE_VISION_MODEL=claude-haiku-4-5

PAID_FALLBACK_MODE=ask_each_job # ask_each_job | auto_when_key_exists | disabled

PAID_FALLBACK_BUDGET_USD_PER_JOB=1.00

Keep keys server-side only. Never expose keys to the browser. Redact keys in logs.

7. Fallback Algorithm

Fallback must be page-level and field-level.

For each document page:

Run local preprocessing and OCR.

Calculate OCR confidence, coverage, and extraction completeness.

If all quality gates pass:

mark page as local_pass.

Else:

mark page as local_low_confidence.

If BYOK keys exist and policy allows paid fallback:

send only failed page/image or field crop to paid provider.

merge paid extraction with local extraction.

record provider, cost estimate, timestamp, and reason.

Else:

continue with warning and degraded quality flag.

Trigger paid fallback when any of these conditions is true:

- OCR confidence < 75.
- More than 20% of page has no recognized text but image is not blank.
- Required field is missing for selected draft type.
- Retrieval returns no evidence for a generated claim.
- Local model returns malformed JSON twice.
- User manually clicks “Improve with paid model”.

8. OCR Quality Metrics

8.1 Metrics to Store

```
{  
  "document_id": "uuid",  
  "page": 1,  
  "provider": "local_ocr|gemini_byok|claude_byok",  
  "ocr_confidence": 87.4,  
  "word_count": 924,  
  "low_confidence_word_count": 51,  
  "page_coverage": 0.93,  
  "field_completeness": 0.91,  
  "fallback_used": false,  
  "quality_band": "green|amber|red"  
}
```

8.2 Dashboard Bands

- Green: OCR confidence ≥ 90 and coverage $\geq 95\%$.
- Amber: OCR confidence 75-89 or coverage 80-94%.
- Red: OCR confidence < 75 or coverage < 80%.

8.3 User-Facing Warning

When quality drops, show:

> Extraction quality is low on pages 2 and 5. Review the highlighted text before using this draft. Paid BYOK fallback is available for these pages.

9. Retrieval and Grounding

9.1 Chunking

- Chunk by page, heading, paragraph, and table boundaries.
- Target chunk size: 500-900 tokens.
- Keep metadata: source file, page, bounding box when available, OCR confidence, extraction provider.

9.2 Retrieval

- Use local embedding model by default.
- Return top-k chunks with reranking.
- Do not draft from memory. Draft only from retrieved evidence.

9.3 Citation Format

Every factual claim must carry source metadata:

```
{  
  "claim": "Tenant received notice on 2026-04-10.",  
  "source": "notice_scan.pdf",  
  "page": 2,  
  "quote_or_span": "Notice dated April 10, 2026",  
  "confidence": 0.91  
}
```

If no evidence exists, place it under “Unsupported or Needs Review.”

10. Draft Generation

Supported outputs:

5. Review summary.
6. Case fact summary.
7. Notice-related summary.
8. Document checklist.
9. First-pass internal memo.

Draft rules:

- Use concise headings.
- Include “Known facts,” “Potential issues,” “Missing information,” and “Source evidence.”
- Never claim legal conclusion unless directly supported.
- Mark uncertainty explicitly.
- Include a reviewer warning when quality band is amber or red.

11. Improvement from Edits

Capture edit events:

```
{  
  "draft_id": "uuid",  
  "before": "original sentence",
```

```
"after": "edited sentence",  
"edit_type": "tone|missing_fact|incorrect_fact|format|citation|clarity",  
"accepted": true,  
"reusable_lesson": "Prefer chronological ordering in case fact summaries."  
}
```

Use edits to update:

- Prompt templates.
- Draft structure preferences.
- Extraction rules.
- Citation validation rules.

Never store sensitive raw snippets in long-term learning unless the admin enables it. Store generalized lessons by default.

12. Data Model

Core tables/collections:

- documents
- pages
- extraction_results
- quality_metrics
- chunks
- retrieval_runs
- drafts
- draft_claims
- citations
- edit_events
- provider_calls
- app_settings

13. API Endpoints

POST /api/documents/upload

GET /api/documents/:id/status

GET /api/documents/:id/extractions

GET /api/documents/:id/quality

POST /api/documents/:id/fallback

POST /api/drafts

GET /api/drafts/:id

PATCH /api/drafts/:id

POST /api/drafts/:id/edits

GET /api/settings/providers

PATCH /api/settings/providers

14. UI Requirements

Pages:

- Upload page.
- Processing status page.
- Extraction review page with low-confidence highlights.
- Quality dashboard.

- Draft generator page.
- Draft review/edit page.
- BYOK provider settings page.

Quality dashboard must show:

- Overall quality band.
- Per-page confidence.
- Fallback usage.
- Unsupported claim count.
- Citation coverage.
- Edit delta after review.

15. Error Handling

- OCR crash: retry once with safer preprocessing, then mark failed.
- Local model unavailable: continue OCR-only or ask for paid fallback.
- Paid API key invalid: show setup error, do not retry repeatedly.
- Rate limit: exponential backoff and show “paid provider delayed.”
- Malformed model JSON: repair once; if still invalid, retry with stricter schema prompt; then fallback.
- Missing citations: block final draft export until user accepts warning.

16. Security and Privacy

- BYOK keys encrypted at rest or stored in local env only.
- No keys in frontend, logs, or exported files.
- Raw document retention configurable.
- Delete job data from UI.
- Basic audit log for fallback calls and edits.
- Redact PII in debug logs.

17. Testing Plan

Unit Tests

- File type validation.
- OCR confidence calculation.
- Provider fallback selection.
- JSON schema validation.
- Citation format validation.
- Edit event classification.

Integration Tests

- Upload PDF -> local OCR -> chunks -> draft.
- Low-confidence scan -> paid fallback path mocked.
- Invalid BYOK key -> graceful error.
- Draft claim with no evidence -> unsupported warning.
- Operator edit -> reusable lesson stored.

End-to-End Tests

- Happy path: clear PDF to review summary.
- Noisy scan path: local fail to paid fallback to draft.
- Missing key path: local fail to degraded dashboard warning.
- Edit learning path: first draft edited, second draft improves formatting.

18. Acceptance Criteria

The project is complete when:

- A user can upload a messy document bundle.
- The app extracts text and structured fields locally.
- The user can see OCR quality and accuracy warnings.
- Low-quality pages can be sent to Gemini or Claude with BYOK keys.
- Generated drafts include citations and unsupported claim warnings.
- User edits are captured and summarized into reusable improvements.
- All tests pass locally.
- README, spec, playbook, and journal are included.

19. Agent Task List

Agent 1: Repo and Config

Create project skeleton, environment config, provider interfaces, logging, and test runner.

Done when:

- Repo runs locally.
- ``.env.example`` exists.
- Provider interface tests pass.

Agent 2: Intake and Normalization

Build upload flow, PDF rendering, image preprocessing, and file metadata storage.

Done when:

- PDF/images produce normalized page images.
- Unsupported files are rejected safely.

Agent 3: OCR and Quality

Build local OCR adapter, confidence calculation, quality dashboard API.

Done when:

- Per-page OCR confidence is stored.
- Low-confidence pages are flagged.

Agent 4: BYOK Paid Fallback

Build Gemini and Claude adapters behind common interface. Add fallback policy.

Done when:

- Mocked paid provider tests pass.
- Invalid key handling works.
- Per-page fallback records are stored.

Agent 5: Retrieval and Grounding

Build chunker, embedding store, retrieval, citation metadata, and claim verifier.

Done when:

- Draft inputs include cited evidence spans.
- Unsupported claims are blocked or flagged.

Agent 6: Draft Generation

Build template-based drafting for five output types.

Done when:

- Drafts have required sections.
- Draft facts cite source evidence.

Agent 7: Edit Learning

Build edit capture, edit classification, reusable lesson store, and prompt adjustment.

Done when:

- Edits are stored and summarized.
- Future drafts can use stored style/structure lessons.

Agent 8: QA and Documentation

Write fixtures, tests, README, playbook, and demo script.

Done when:

- Unit/integration/E2E tests pass.
- Demo can be run from README.

20. Questions for Owner Before Coding

10. Should the demo focus on legal notice summaries, case fact summaries, or first-pass memos?
11. Should paid fallback be automatic when API keys exist, or always ask first?
12. What languages must OCR support on day one: English only, Bengali only, or mixed?
13. Is Docker allowed in the office submission environment?
14. Should document data be deleted after each demo run by default?

Document 3: Build Journal

Decision log and work narrative

Build Journal

Project: Local-First Document Understanding and Grounded Drafting

Journal Entry 001 - Requirement Intake

The assessment asks for a system that can ingest messy legal-style documents, extract useful content, retrieve relevant evidence, generate grounded drafts, and improve from operator edits. The primary risk is not generation; it is building enough trust in the extraction, retrieval, and citation chain.

Decision: Build a local-first architecture with paid BYOK fallback. This satisfies cost control, privacy, and reliability.

Journal Entry 002 - Scope Selection

Selected MVP outputs:

- Review summary.
- Case fact summary.
- Notice-related summary.
- Document checklist.
- First-pass memo.

Reason: These outputs are explicitly aligned with the assessment and are practical for demo review.

Journal Entry 003 - Local-First Design

The system will run OCR and extraction locally first. Local failure does not mean the whole job fails. Instead, the system marks the page or field as low-confidence and optionally escalates only that portion to a paid model.

Reason: Page-level fallback is cheaper and easier to justify than sending the full file to a paid model every time.

Journal Entry 004 - BYOK Decision

Gemini and Claude vision adapters will be configured using user-owned API keys. Keys are never exposed to the browser and are redacted from logs.

Reason: The office can test paid performance without the developer owning API spend or data routing decisions.

Journal Entry 005 - Quality Metrics

Quality is not hidden. The user sees OCR confidence, page coverage, field completeness, fallback usage, citation coverage, unsupported claim count, and edit delta.

Reason: This addresses the rubric's emphasis on OCR quality, grounding, and improvement from edits.

Journal Entry 006 - Grounding Decision

Drafts must be generated only from retrieved evidence. Unsupported facts go into a dedicated warning section.

Reason: The assessment explicitly says the system should not hallucinate unsupported content.

Journal Entry 007 - Edit Improvement Loop

The first implementation will capture edits and summarize them into reusable lessons. It will not train a new model.

Reason: Prompt/template improvement is safer and realistic for an MVP.

Journal Entry 008 - Testing Strategy

Use synthetic but realistic fixtures: clear PDF, noisy scan, handwritten note, mixed screenshot, missing key fallback, invalid key fallback, unsupported claim, and edit improvement.

Reason: Synthetic data avoids sensitive material while proving the workflow.

Journal Entry 009 - Implementation Order

Build order:

15. Repo/config.
16. Intake/normalization.
17. OCR/quality.
18. BYOK fallback.
19. Retrieval/grounding.
20. Drafting.
21. Edit learning.
22. Tests/docs/demo.

Reason: This order makes every later stage testable and prevents generation from being built before extraction is reliable.

Journal Entry 010 - Pre-Coding Approval Questions

Before coding, confirm:

- Default demo draft type.
- Auto vs manual paid fallback.
- Day-one OCR languages.
- Local runtime allowed in the office environment.
- Data retention policy for demo files.

Document 4: Basic Playbook

Run, demo, QA, and troubleshooting guide

Basic Playbook

1. Purpose

This playbook explains how to run, test, demo, and troubleshoot the local-first document understanding system.

2. Local Setup

Expected setup after implementation:

```
cp .env.example .env
```

Fill optional BYOK keys only if paid fallback is needed.

```
npm install
```

```
npm run dev
```

Optional local model setup:

Example only; final model can be changed before coding.

```
ollama pull qwen2.5vl
```

```
ollama pull llama3.2
```

3. BYOK Setup

Add keys server-side:

```
GEMINI_API_KEY=your_key_here
```

```
GEMINI_VISION_MODEL=gemini-3-flash
```

```
ANTHROPIC_API_KEY=your_key_here
```

```
CLAUDE_VISION_MODEL=claude-haiku-4-5
```

```
PAID_FALLBACK_MODE=ask_each_job
```

Recommended default: `ask\_each\_job` so the operator controls paid usage.

4. Demo Script

23. Open app.
24. Upload a clear PDF and a noisy scan.
25. Choose “Case Fact Summary.”
26. Watch local OCR processing complete.
27. Open the quality dashboard.
28. Show green/amber/red page quality.
29. Trigger paid fallback for the low-confidence page.
30. Generate grounded draft.
31. Click citations to inspect evidence.
32. Edit one paragraph.
33. Show edit lesson captured.

34. Regenerate or start a second draft using the improvement lesson.

5. Quality Review Checklist

Before trusting output, verify:

- OCR confidence is green or amber with review completed.
- Required fields are present.
- Every major factual claim has a citation.
- Unsupported claims are listed separately.
- Paid fallback usage is visible.
- The draft includes missing information and reviewer warnings.

6. Troubleshooting

OCR confidence is low

- Check scan rotation.
- Try image preprocessing.
- Use paid BYOK fallback for failed pages.
- Manually correct extracted text if needed.

Paid model does not run

- Check API key exists server-side.
- Check model name.
- Check fallback mode is not disabled.
- Check rate limit or billing status.

Draft has unsupported claims

- Re-run retrieval.
- Reduce draft scope.
- Add the claim to “Unsupported or Needs Review.”
- Do not export as clean final without accepting warning.

Citations point to wrong page

- Rebuild chunks.
- Check PDF page rendering order.
- Check OCR metadata page numbers.

User edits are not improving output

- Confirm edit events are being stored.
- Confirm reusable lessons are being applied to the selected draft template.
- Review edit classification labels.

7. Release Checklist

- All tests pass.
- `.env.example` has no secrets.
- README explains local setup.
- Sample fixtures are synthetic.
- Demo data can be deleted.
- Fallback provider logs redact keys.
- Quality dashboard shows OCR accuracy clearly.
- Journal and spec files are included.